



Installing a CNI Plugin and Validating Cluster Health



Tim Warner

Principal Author | IT Ops | Pluralsight

TechTrainerTim.com



Four Framing Questions



Why do nodes stay NotReady until a CNI plugin is installed?



How does Calico compare to other CNI plugins for CKA prep?



What is the cluster validation checklist after CNI installation?



How do I diagnose common kubeadm bootstrap failures?





“Three nodes, all NotReady. CoreDNS Pending.
The control plane is fine — we just need a CNI plugin.
Get the CIDR right, or we'll waste time debugging
silent, invisible network failures.”



Why do nodes stay NotReady until a CNI plugin is installed?



The CNI specification and why nodes need it

CNI assigns IP addresses and configures networking for Pods

Without a CNI, kubelet cannot assign Pod IPs — nodes remain NotReady

CoreDNS depends on pod networking — it cannot start without CNI

CNI plugins run as DaemonSets — one per node in the cluster

Kubernetes delegates networking to plugins — CNI defines the interface



Comparing CNI plugins: Calico, Flannel, and Cilium



Calico

Supports NetworkPolicy

Common in self-managed clusters

Strong default for CKA practice



Flannel

Lightweight and simple

Good for basic pod networking

No native NetworkPolicy



Cilium

Advanced eBPF networking

Strong visibility and policy features

More complexity than most CKA labs need



**What is the cluster validation checklist
after CNI installation?**



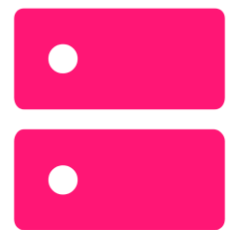
The six-point cluster validation checklist



All nodes show Ready in `kubectl get nodes`



All kube-system pods show Running (CoreDNS transitions from Pending)



CoreDNS resolves names: `nslookup kubernetes.default` from a `busybox:1.28` test pod (exam-safe pin)



The six-point cluster validation checklist



Cross-node pod communication works: ping between pods on different nodes



kubectl cluster-info shows healthy API server and CoreDNS endpoints

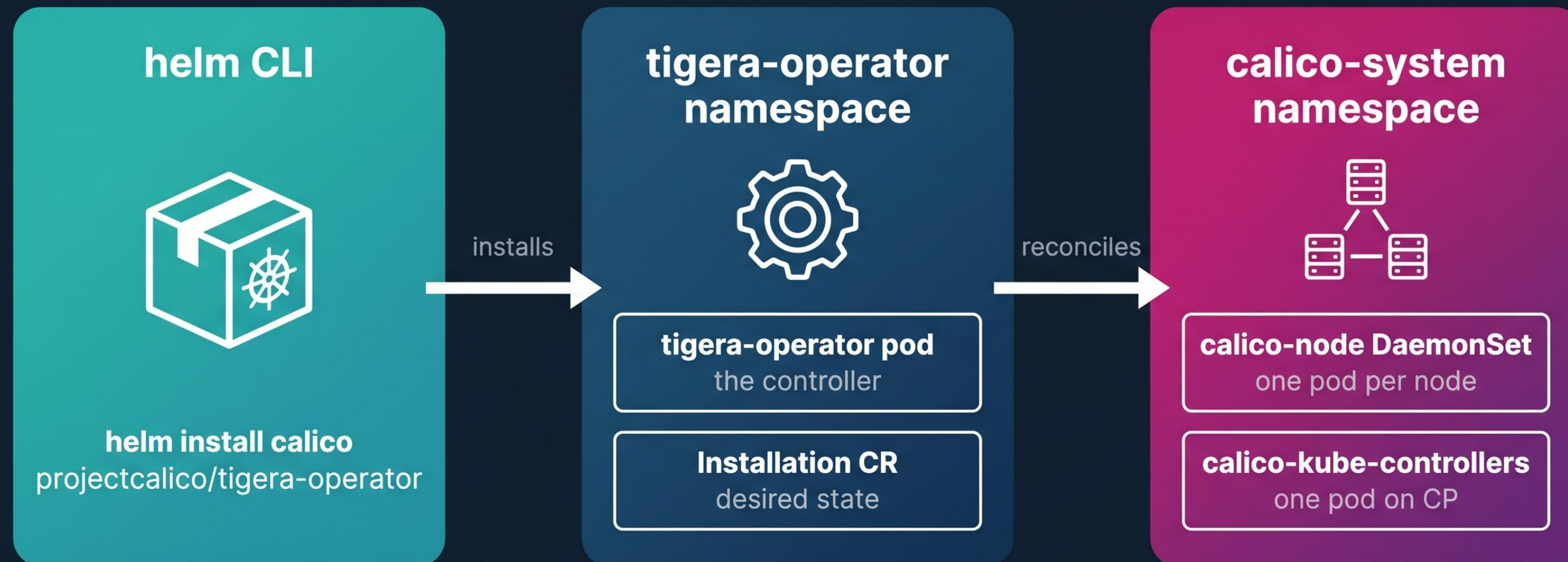


kubectl get pods -A shows no CrashLoopBackOff or Pending (cluster stable)



Inside the Tigera operator: how Calico lands on every node

Helm install → Tigera operator → Calico on every node



Operator pattern — declare desired state, the operator makes it real.



Helm-install Calico, then prove DNS works (the only validation that matters)

```
# Install Calico via Helm + tigera-operator (v1.35 objective: Helm/Kustomize for cluster components)
helm repo add projectcalico https://docs.tigera.io/calico/charts
helm repo update
helm install calico projectcalico/tigera-operator --version v3.32.0 \
  --namespace tigera-operator --create-namespace

# Watch nodes go NotReady -> Ready (Calico DaemonSet starting on each node)
kubectl get nodes -w
kubectl -n calico-system get pods -w      # operator-managed Calico pods

# Verify CoreDNS Pending -> Running
kubectl -n kube-system get pods | grep coredns

# DNS smoke test -- the real "is the cluster usable" check (busybox:1.28 = exam-safe pin)
kubectl run dns-test --image=busybox:1.28 --rm -it --restart=Never -- nslookup kubernetes.default

# Expected output ends with: Address: 10.96.0.1 kubernetes.default.svc.cluster.local
```



How do I diagnose common kubeadm bootstrap failures?



Diagnosing kubeadm bootstrap failures

kubelet layer:
journalctl -u kubelet
-f (live tail)

Runtime layer: crictl
info + crictl ps

Certificate issues:
kubeadm certs check-
expiration for cert
status

Node NotReady after
CNI: kubectl describe
node for conditions
and events

Expanded ladder: get
-> describe -> logs ->
events -> journalctl -
> crictl ps



Demo

Install Calico, validate the cluster, and practice failure diagnosis





“Cluster is green across the board. Nodes Ready, CoreDNS resolving, pods communicating across nodes, and nginx serving traffic through a NodePort. We broke it, diagnosed it, and fixed it. This cluster is ready for workloads.”



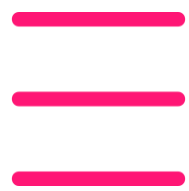


From Globomantics to You



CNI is what makes nodes Ready

Without a CNI plugin, kubelet cannot assign pod IPs and the cluster stays non-functional



Calico is the CKA default for a reason

NetworkPolicy support for Courses 7-8, compatible CIDRs, and active maintenance



Validate every layer after infrastructure changes

Nodes Ready, pods Running, DNS resolving, cross-node connectivity, workloads serving



Break it on purpose to build troubleshooting muscle

journalctl, crictl, and the diagnostic ladder are your exam-day tools





Course summary

Infrastructure prep is non-negotiable: kernel modules, sysctl, containerd, version pinning

kubeadm init creates certificates, static pods, and kubeconfig in one command

CNI installation transforms a bootstrapped cluster into a functional one

The diagnostic ladder (get, describe, logs, events) applies to every failure scenario





Thank you for your time!

Tim Warner | TechTrainerTim.com

